

PARTE 1 - Classe Base e Herança

Exercício 1 - Classe Abstrata

Crie a classe abstrata:

Forma

- **Atributos:**
 - `std::string nome;`
- **Métodos:**
 - `virtual ~Forma() = default;`
 - `virtual double area() const = 0;`
 - `virtual double perimetro() const = 0;`
 - `virtual std::string descricao() const;` - retorna a string:

“[NomeDaForma] | Area: X.XX | Perimetro: Y.YY”

Exercício 2 — Classes Derivadas

Crie essas classes derivadas de **Forma**:

Retangulo

- **Atributos:**
 - `double largura, double altura;`
- **Métodos:**
 - `Retangulo(const std::string& nome, double largura, double altura);`
 - `double area() const;`
 - `double perimetro() const;`
 - `bool quadrado() const;`
 - sobrescreva `std::string descricao() const;`

Circulo

- **Atributos:**
 - `double raio;`
- **Metodos:**
 - `Circulo(const std::string& nome, double raio);`
 - `double area() const;`
 - `double perimetro() const;`
 - sobrescreva `std::string descricao() const;`

Obs:

- **implemente tratamento de erros para:**
 - construtor com raio, largura ou altura negativos

PARTE 2 - Sobrecarga de Operadores

Exercício 3 - Retângulo

Implemente:

- `operator+` - soma dimensões
- `operator*` e `operator*=-` - escala
- `operator==` - compara área
- `operator<` e `operator>` - compara área
- `operator<<` - retorna `descricao()` para o output

Exercício 4 - Círculo

Implemente:

- `operator+` - soma raios
- `operator*` e `operator*=-` - escala
- `operator==` - compara área
- `operator<<` - retorna `descricao()` para o output

Obs: implemente tratamento de erros para:

- escala com valores negativos
- soma com valores que resulte em raio, altura ou largura ≤ 0

PARTE 3 - Templates

Exercício 5 - Classe Template

Crie:

```
template<typename T, typename U>  
class Par
```

- **Atributos:**

- T primeiro;
- U segundo;

- **Métodos:**

- Par(T primeiro, U segundo);
- T getPrimeiro() const;
- U getSegundo() const;
- void setPrimeiro(T valor);
- void setSegundo (U valor);
- Par<U,T> invertido() const; - retorna um novo **Par** com os tipos e valores invertidos
- operator<< - retorna "(" + getPrimeiro() + ", " + getSegundo() + ")"

PARTE 4 - Testes (main)

Implemente estes testes:

Teste 1 — Polimorfismo

```
std::vector<Forma*> formas;
formas.push_back(new Retangulo("R", 4, 3));
formas.push_back(new Circulo("C", 2));

for (auto f : formas)
    std::cout << f->descricao() << std::endl;

for (auto f : formas)
    delete f;
formas.clear();
```

Teste 2 — Operadores

```
Retangulo r1("R1", 2, 3);
Retangulo r2("R2", 1, 1);

r1 *= 2.5;

std::cout << r1 << std::endl;
std::cout << (r1 + r2) << std::endl;
std::cout << (r1 * 2) << std::endl;
```

Teste 3 — Template

```
// Par<Retangulo, Circulo>
Retangulo r1("R1", 3, 4);
Circulo c1("C1", 5);

Par<Retangulo, Circulo> p1(r1, c1);
std::cout << p1 << std::endl;

// invertido() gera Par<Circulo, Retangulo>
Par<Circulo, Retangulo> p2 = p1.invertido();
std::cout << p2 << std::endl;
```

Teste 4 — Exceção

```
Retangulo r2("R2", 1, 1);
r2 *= -1;

Circulo c1("C1", -5);
```